

UNITED STATES PATENT AND TRADEMARK OFFICE

(USPTO)

PROVISIONAL PATENT APPLICATION SPECIFICATION

TITLE OF THE INVENTION

****SYSTEM AND METHOD FOR REDUCING NETWORK LATENCY AND ELIMINATING DISTRIBUTED CONSENSUS BOTTLENECKS IN ASYNCHRONOUS MACHINE-TO-MACHINE STREAMING SETTLEMENTS AND COLLATERAL FORECLOSURE****

INVENTOR(S)

- ****Inventor Name:**** [Insert Full Legal Name / ФИО Заявителя]
 - ****Citizenship / Residence:**** [Insert Country / City]
-

CROSS-REFERENCE TO RELATED APPLICATIONS

- Not applicable. This application is an original provisional filing under 35 U.S.C. § 111(b).
-

STATEMENT REGARDING FEDERALLY SPONSORED RESEARCH OR DEVELOPMENT

- None.
-

FIELD OF THE INVENTION

The present invention relates generally to distributed computing systems, asynchronous machine-to-machine (M2M) communications, and decentralized ledger architectures. More particularly, the invention relates to computer-implemented methods and systems for drastically reducing network latency and memory bottlenecks in high-frequency computational micro-transactions between client computing processes, featuring deterministic key-exposure equivocation traps, pipelined micro-exposure bounds, and automated collateral foreclosure under perfected commercial security agreements.

BACKGROUND OF THE INVENTION AND PRIOR ART

Autonomous artificial intelligence (AI) agents (e.g., executing on LangGraph, AutoGen, CrewAI, or decentralized compute clusters) increasingly require high-frequency, ephemeral micro-transactions to purchase incremental computational services, such as large language model (LLM) inference tokens, vector similarity searches, scraping bandwidth, and GPU execution cycles. The monetary value of such individual computational micro-tasks typically ranges from fractions of a cent (\$0.0001) to several cents (\$0.05).

Existing financial and cryptographic infrastructure suffers from fundamental technical bottlenecks that render them economically or computationally unviable for autonomous agent commerce:

1. **Web2 Centralized Payment Rails (Stripe, Visa, Mastercard):** Impose fixed baseline interchange fees (e.g., \$0.30 plus 2.9% per transaction). For a \$0.001 micro-service call, a \$0.30 fee represents an unsustainable 30,000% overhead. Furthermore, Web2 rails require human identity verification (KYC/KYB), credit cards, and manual chargeback dispute processes incompatible with programmatic, autonomous M2M workflows.
2. **Blockchain Layer-1 (L1) and Layer-2 (L2) Networks (Ethereum, Solana, Arbitrum, Base):** Require every state transition to be validated, sequenced, and committed into distributed blocks. This introduces unavoidable network latency (400 milliseconds to 12 seconds) and per-transaction gas fees (\$0.001 to \$0.05). For continuous token streaming at hundreds of queries per second, cumulative gas costs exceed the value of the underlying compute, and sequencer throughput bottlenecks lead to mempool congestion.
3. **Bilateral Payment Channel Networks (Lightning Network, Raiden):** Require bidirectional

locked collateral along every hop of a routed path (Hashed Time-Locked Contracts — HTLCs). This architecture causes severe capital fragmentation, where routing failure rates exceed 30% for dynamic multi-hop payments, and continuous channel rebalancing incurs prohibitive L1 transaction fees.

4. **Prior Key-Exposure Schemes (Stefan Brands e-cash, Babylon EOTS):** While theoretical Extractable One-Time Signatures (EOTS) have been utilized for consensus validator slashing in Proof-of-Stake blockchains (e.g., signing a block once every 10 seconds), prior art fails to provide a scalable, non-blocking mechanism for *asynchronous, high-frequency, multi-vendor streaming micro-payments* without risking catastrophic collateral overdraft or severe liquidity lockups.

Accordingly, there is an urgent and unfulfilled need for a trustless, zero-gas, sub-millisecond peer-to-peer settlement architecture that provides mathematical finality and fraud prevention for autonomous software processes without requiring monolithic on-chain execution for every micro-transaction.

SUMMARY OF THE INVENTION

The present invention provides a novel cryptoeconomic architecture and protocol, termed **Causal-Slash**, that eliminates the blockchain from intermediate transactional messaging while preserving L1-level settlement guarantees.

In an exemplary embodiment, an autonomous computational agent deposits collateral into a decentralized performance collateral vault (governed as a perfected security interest under Article 9 of the Uniform Commercial Code — UCC Article 9) on a base blockchain (L1/L2). The agent generates a sequence of deterministic nonces bound to a strictly monotonic local state variable h in $\mathbb{N}$ using a keyed pseudorandom function (e.g., HMAC-SHA256). The public commitments of these nonces are structured as leaves of a cryptographic Merkle tree, whose root is anchored on-chain.

When transmitting micro-payments directly peer-to-peer (P2P) over transport layer protocols (HTTP/WebSocket/QUIC), the agent issues an Extractable One-Time Signature (EOTS) cheque for height h . If the agent attempts a double-spending equivocation (issuing two conflicting cheques with different messages $m_1 \neq m_2$ at the identical state height h), the algebraic structure of the Schnorr signature scalar allows any observer to solve a system of linear equations using localized modular arithmetic operations, extracting the agent's secret key in sub-microsecond time:

$$s_k = (s_1 - s_2) \cdot (e_1 - e_2)^{-1} \pmod{q}$$

To prevent multi-vendor overdraft attacks (where an agent concurrently drains multiple vendors before a foreclosure transaction finalizes), the invention implements a **Pipelined Streaming Micro-Exposure Invariant**. Services and cheques are interleaved such that each vendor maintains a localized un-settled exposure cap $\delta_v \leq \$1.00$. The required collateral parameter B is mathematically constrained to exceed the aggregate concurrent exposure across all connected vendors ($B > \sum \delta_v$), ensuring that any equivocation attack results in a mathematically guaranteed net financial loss to the attacker.

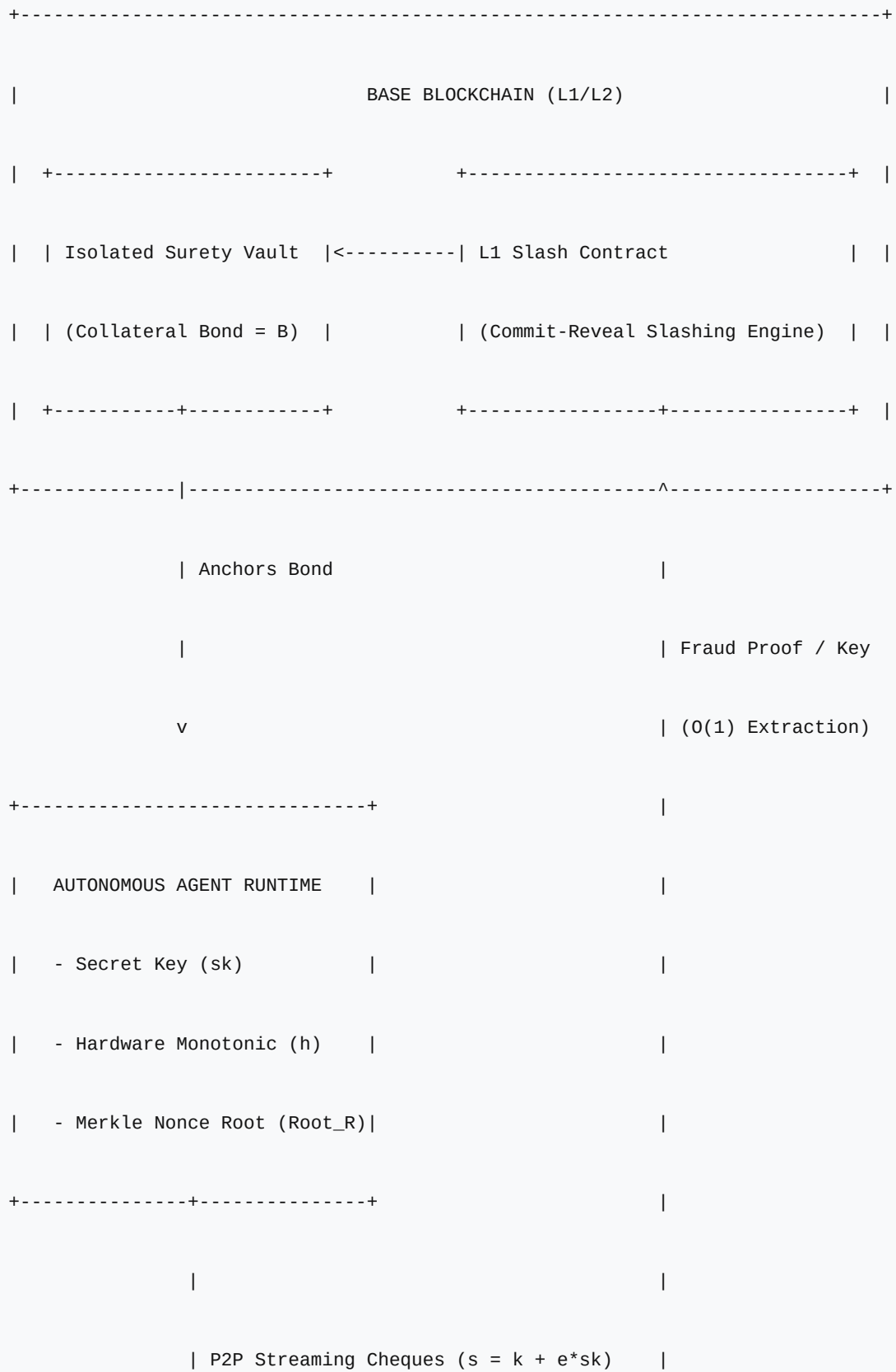
Furthermore, the invention features:

1. **Two-Phase Commit-Reveal Slashing:** Protects fraud-proof submitters from mempool front-running by maximal extractable value (MEV) searchers.
2. **Graduated Liquidated Damages:** Proportional collateral foreclosure scaling as $\min(B, 2 \times \text{Damage})$, preventing accidental loss of entire collateral pools due to software concurrency errors.
3. **Multilateral Peer-to-Peer Debt Graph Cancellation:** Resolves cyclic debt off-chain through directed graph cycle cancellation, reducing periodic L1 settlement overhead by over 60%.

BRIEF DESCRIPTION OF THE DRAWINGS

- **FIG. 1** illustrates the overall system architecture, showing the interaction between the Autonomous Agent, Isolated Surety Vault, P2P Resource Providers, and L1 Base Settlement Contract.
- **FIG. 2** depicts the cryptographic Merkle Nonce-Tree and the generation of deterministic EOTS signatures.
- **FIG. 3** is a sequence diagram illustrating algebraic private key extraction upon detection of height equivocation.
- **FIG. 4** is a flowchart depicting the Pipelined Streaming Micro-Exposure mechanism and local vendor buffer enforcement.
- **FIG. 5** illustrates the Two-Phase Commit-Reveal Slashing mechanism against MEV mempool front-running.

FIG. 1: OVERALL SYSTEM TOPOLOGY



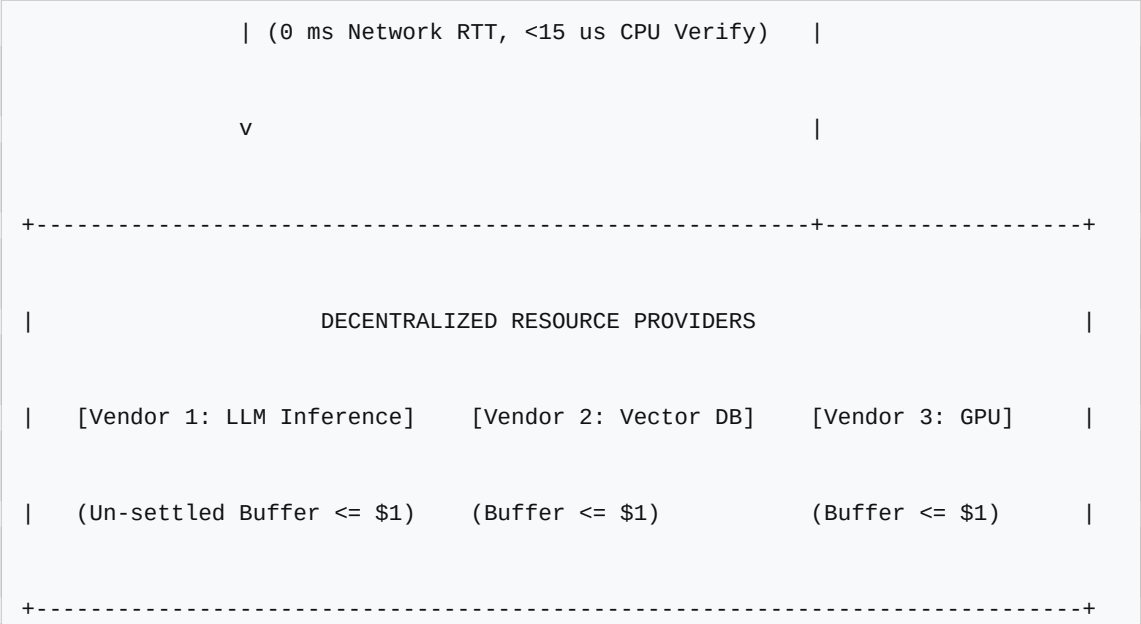
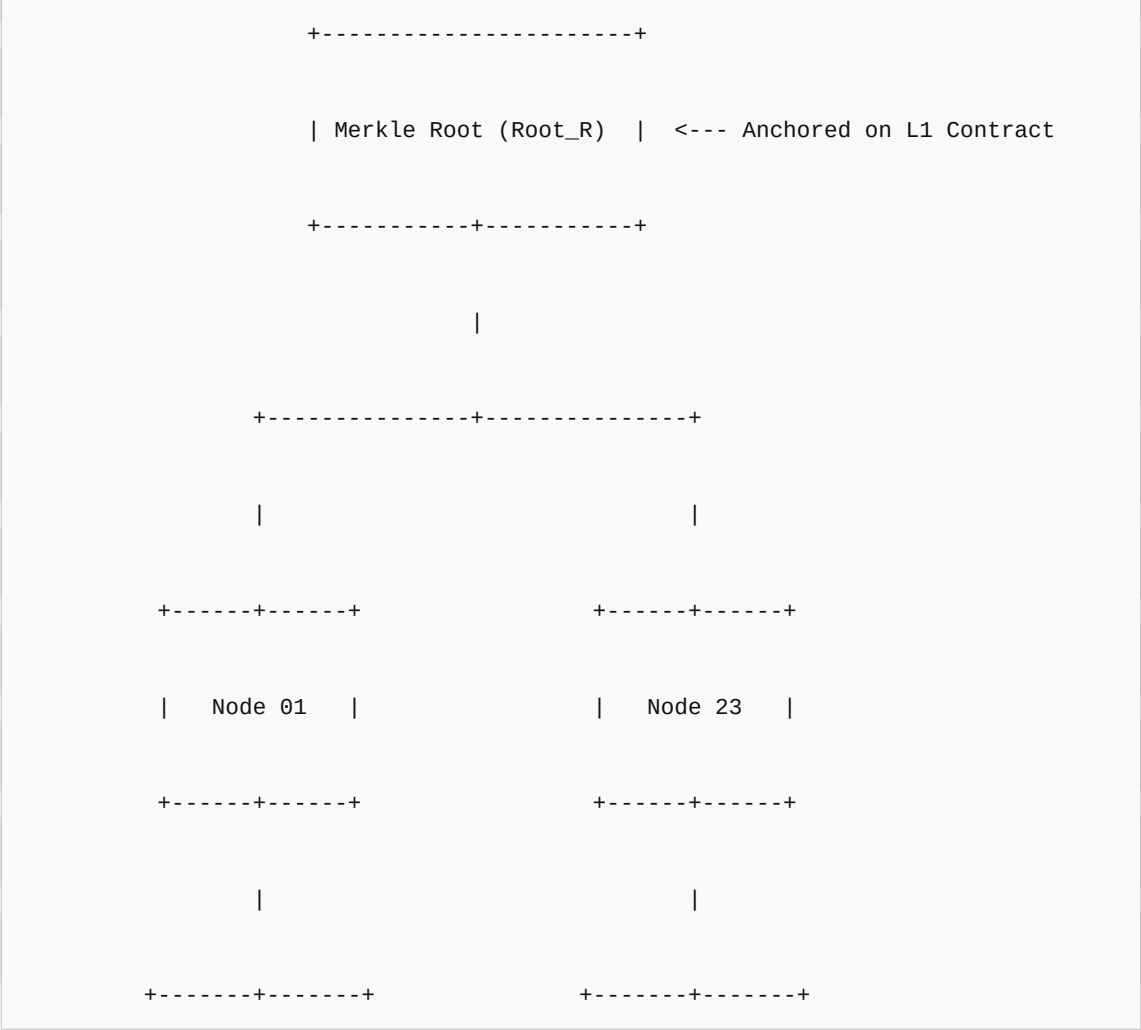


FIG. 2: CRYPTOGRAPHIC MERKLE NONCE-TREE TOPOLOGY



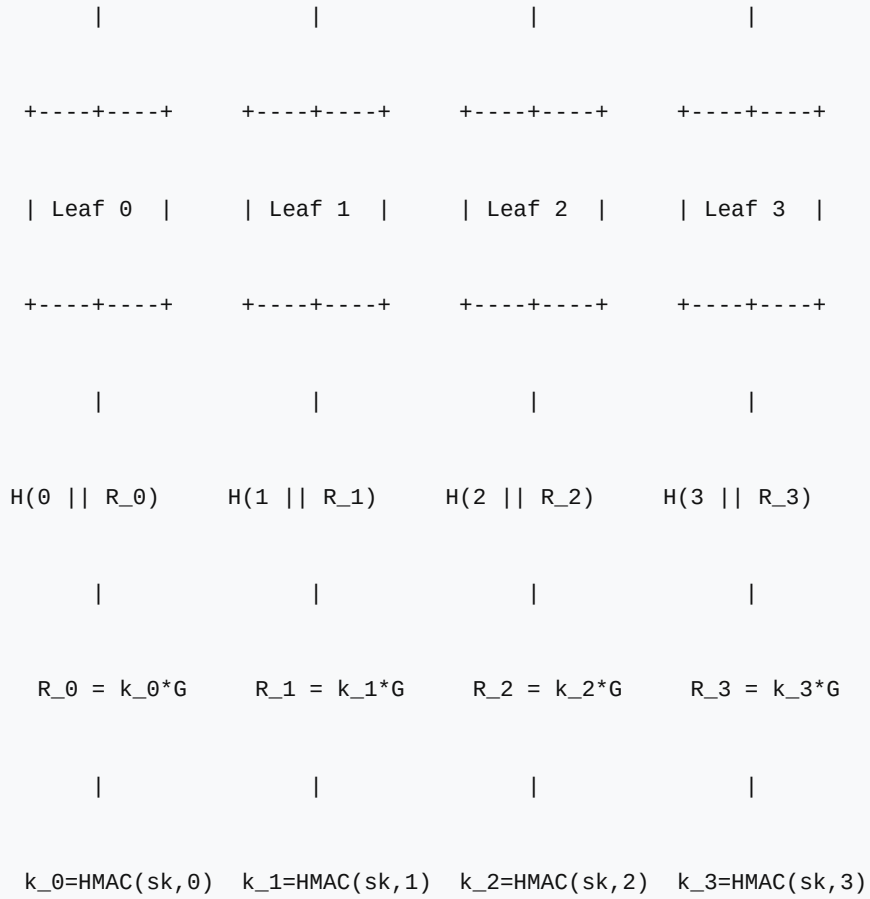
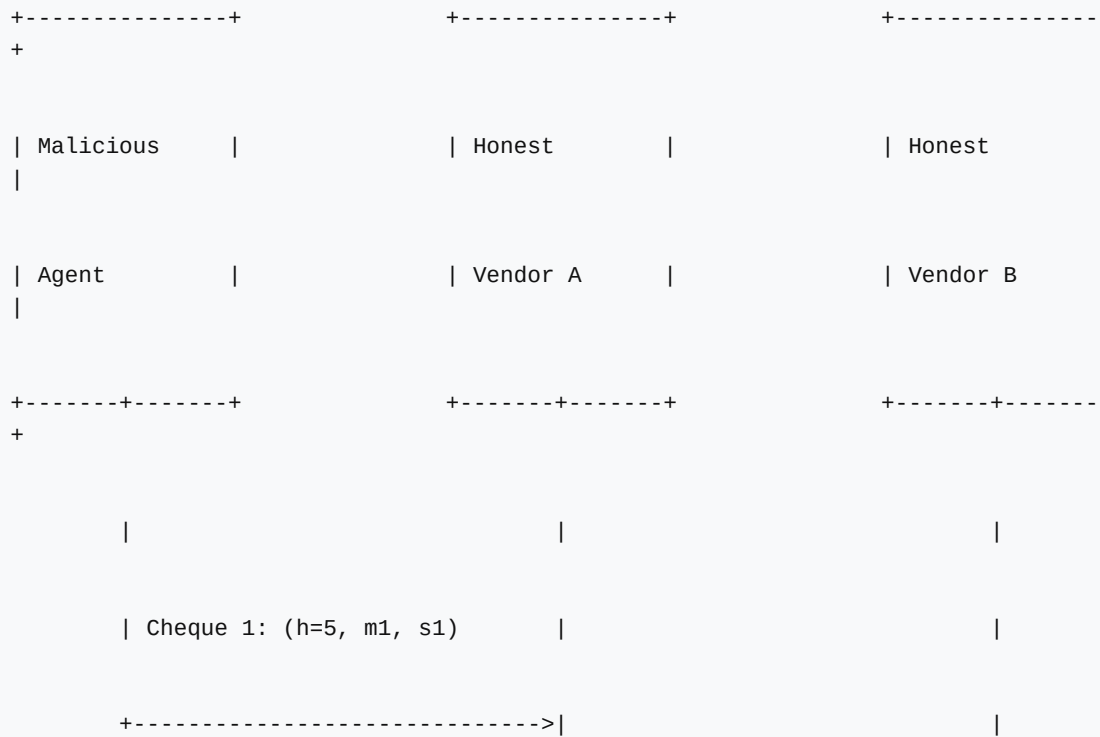
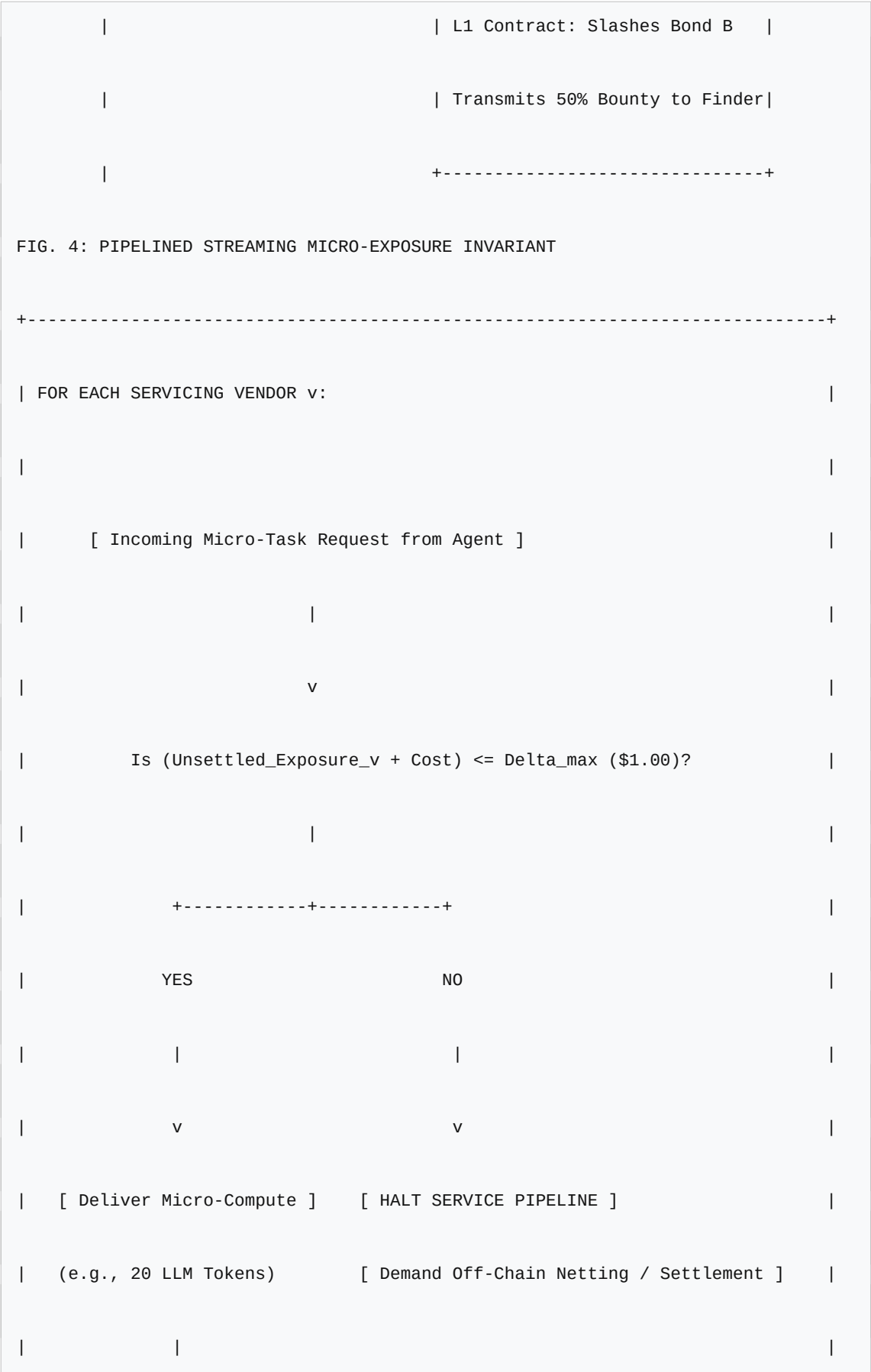


FIG. 3: SUB-MICROSECOND EQUIVOCATION KEY EXTRACTION SEQUENCE





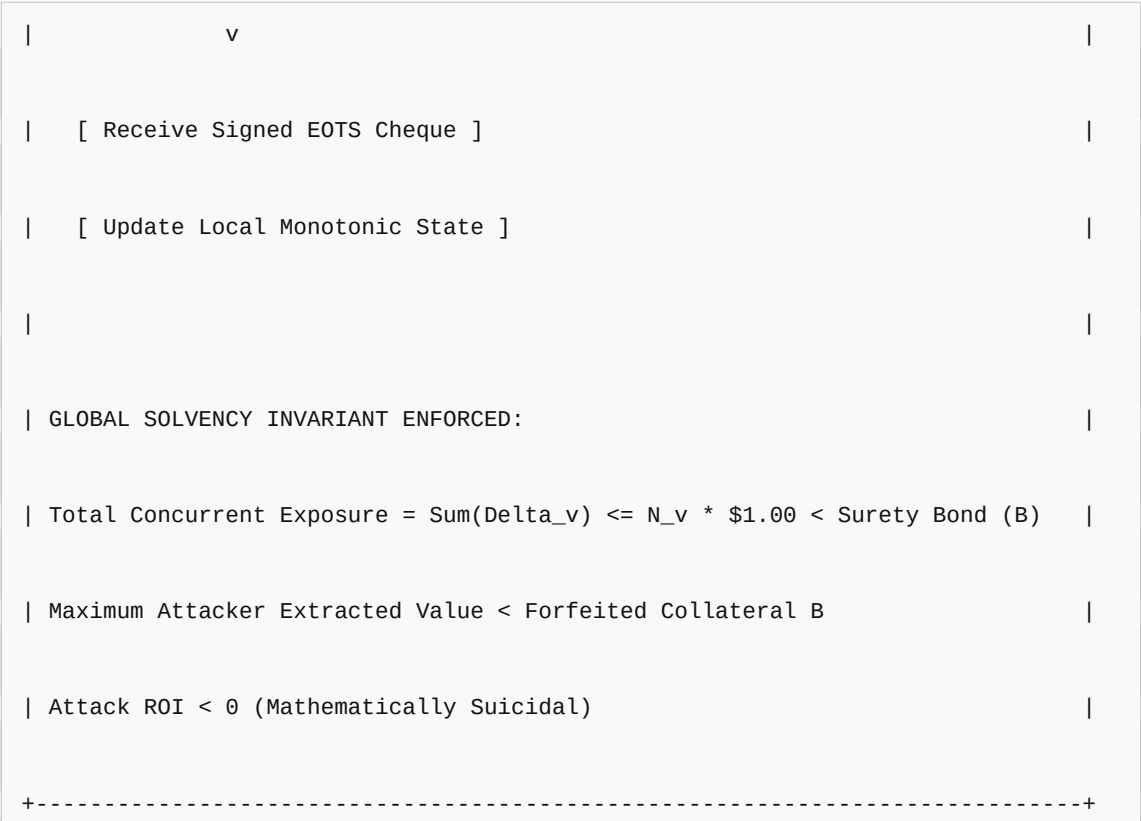
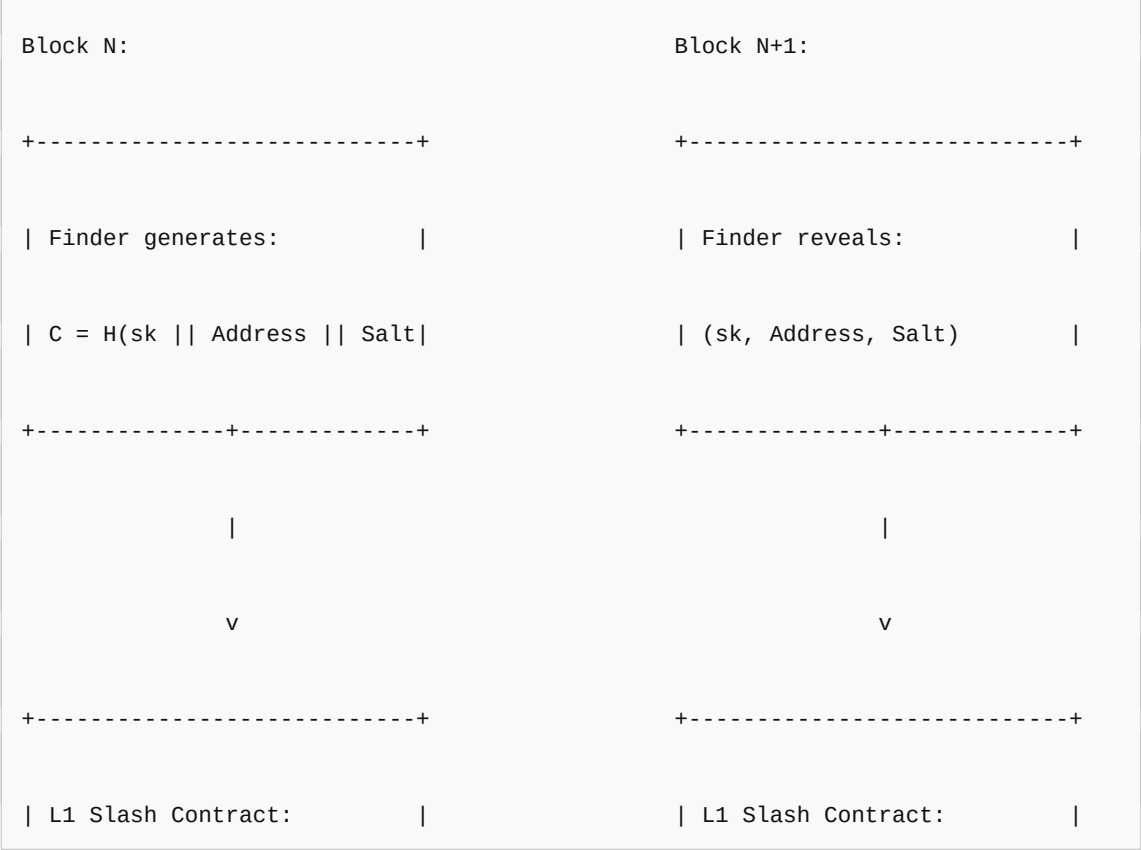


FIG. 5: TWO-PHASE COMMIT-REVEAL SLASHING (MEV FRONT-RUNNING IMMUNITY)



Records Commit(C, Block N)	1. Verify H(sk Addr Salt)
(Mev bots cannot read sk)	2. Verify sk * G == PK
+-----+	3. Slash bond & pay bounty
	exclusively to Address
+-----+	+-----+

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

1. Cryptographic Setup and Deterministic Nonce Tree

Let $\mathbb{G}$ be an elliptic curve group of prime order q with base point generator G (e.g., secp256k1 or Ed25519).

An agent generates a master secret key $sk \in \mathbb{Z}_q$ and public key $PK = sk \cdot G$.

For a sequence of N operational transaction heights $h \in \{0, 1, \dots, N-1\}$, the agent generates deterministic nonces:

$$k_h = \text{HMAC-SHA256}(sk, h) \bmod q$$

$$R_h = k_h \cdot G$$

The agent constructs a Merkle tree $\mathcal{T}$ over the leaf set $\mathcal{L} = \{H(h \parallel R_h)\}_{h=0}^{N-1}$, obtaining Merkle Root Root_R . The agent deposits a surety collateral bond B into the L1 contract and anchors Root_R .

2. P2P Micro-Cheque Issuance and In-Band Verification

To pay an amount a_h to vendor V for a micro-task at height h :

1. The message payload is constructed: $M_h = (\text{AgentID} \parallel V \parallel a_h \parallel h \parallel C_h)$, where $C_h = C_{h-1} + a_h$ is the cumulative spend.

2. The challenge scalar is computed: $e_h = H(R_h \parallel PK \parallel M_h) \bmod q$.

3. The Schnorr scalar signature is generated:

$$s_h = (k_h + e_h \cdot sk) \pmod q$$

4. The tuple $(h, a_h, C_h, R_h, s_h, \text{MerklePath}_h)$ is transmitted directly to vendor V over an existing TCP/QUIC stream.

5. Vendor V validates:

- MerklePath_h proves R_h is committed under Root_R .
- $s_h \cdot G == R_h + e_h \cdot PK$.
- $C_h \leq B$.

Verification requires zero external network round-trips and executes in under 25 microseconds of CPU time.

3. Sub-Microsecond Key Extraction Under Equivocation

If an agent issues two distinct cheques (M_1, R_1, s_1) and (M_2, R_2, s_2) for the identical height h such that $M_1 \neq M_2$:

Since h is identical, the pre-committed nonce requirement forces $R_1 = R_2 = R_h$ and $k_1 = k_2 = k_h$.

The challenges differ: $e_1 = H(R_h \parallel PK \parallel M_1) \neq e_2 = H(R_h \parallel PK \parallel M_2)$.

The signatures satisfy:

$$s_1 = k_h + e_1 \cdot sk \pmod q$$

$$s_2 = k_h + e_2 \cdot sk \pmod q$$

Subtracting the two equations yields:

$$s_1 - s_2 = (e_1 - e_2) \cdot sk \pmod q$$

Because $e_1 \neq e_2$ and q is prime, $(e_1 - e_2)$ has a unique modular inverse in $\mathbb{Z}_q$.

The secret key sk is extracted algebraically in $O(1)$:

$$s_k = (s_1 - s_2) \cdot (e_1 - e_2)^{-1} \pmod{q}$$

Execution of this operation on standard x86_64 architecture takes under 20 microseconds.

4. Pipelined Streaming Micro-Exposure Invariant

To guarantee mathematical solvency across N_v concurrent vendors without global synchronization:

Each vendor v meters service delivery incrementally (e.g., in chunks of $\Delta c = \$0.001$).

Vendor v halts execution if the un-settled, un-cleared exposure exceeds local buffer threshold Δ_v (where $\Delta_v \leq \$1.00$).

The maximum aggregate value extractable by a malicious agent attempting simultaneous forking across all connected vendors is strictly bounded:

$$\text{ExtractableValue}_{\max} = \sum_{v=1}^{N_v} \Delta_v$$

The required surety collateral bond B is enforced to satisfy:

$$B > \sum_{v=1}^{N_v} \Delta_v$$

Consequently, the net economic payoff of any equivocation attack is strictly negative:

$$\mathbb{E}[\text{Payoff}] = \sum_{v=1}^{N_v} \Delta_v - B < 0$$

Rendering multi-vendor draining attacks economically irrational and mathematically suicidal.

5. Two-Phase Commit-Reveal Slashing

To prevent mempool front-running by MEV bots:

- **Phase 1 (Commit):** Finder submits $C = H(sk_{\text{extracted}} \parallel \text{FinderAddress} \parallel \text{salt})$ at block N .
- **Phase 2 (Reveal):** At block $N+1$, Finder reveals $(sk_{\text{extracted}}, \text{salt})$.

The contract verifies that $sk_{\text{extracted}} \cdot G == PK$ and that the hash matches Phase 1. The contract pays the slashing reward exclusively to FinderAddress .

CLAIMS

****We claim:****

1. A computer-implemented method for reducing network latency and eliminating distributed consensus memory bottlenecks in asynchronous machine-to-machine streaming communications, the method comprising:
 - (a) initializing a global performance collateral parameter B within a distributed ledger computing system under a perfected security interest, and anchoring a Merkle root representing a sequence of pre-computed deterministic cryptographic nonces bound to a monotonically strictly increasing hardware state variable h ;
 - (b) executing, by a service provider hardware processor, a localized pipelined service loop comprising incrementally delivering computational resources to a client computing process and asynchronously receiving cryptographic micro-cheques over a peer-to-peer transport layer, bypassing the distributed ledger;
 - (c) enforcing, within a local volatile memory of the service provider hardware processor, a localized un-settled exposure threshold Δ_v , wherein execution of the localized pipelined service loop is automatically halted if an aggregate value of un-settled micro-cheques exceeds Δ_v , and wherein the global performance collateral parameter B is mathematically constrained to strictly exceed an aggregation of all active localized exposure thresholds ($\sum \Delta_v$) across a plurality of independent service providers;
 - (d) detecting, by the service provider hardware processor, a cryptographic equivocation event comprising two conflicting micro-cheques sharing an identical state variable h ; and
 - (e) responsive to the detecting, automatically extracting a private key of the client computing process by executing localized modular arithmetic subtraction and inversion operations on the conflicting micro-cheques, and generating a fraud-proof network transaction incorporating the extracted private key to autonomously trigger execution of a conditional collateral foreclosure penalty against the global performance collateral parameter B on the distributed ledger.
2. The method of claim 1, wherein the cryptographic commitment to the sequence of deterministic nonces comprises a Merkle tree root constructed over leaves comprising hashes of each state variable and corresponding public nonce point $(h \parallel R_h)$.
3. The method of claim 1, wherein the conditional collateral foreclosure penalty on the distributed

ledger disburses pre-agreed liquidated damages to the service provider corresponding to un-settled computational services rendered.

4. The method of claim 1, wherein submitting the fraud-proof network transaction to the distributed ledger comprises a two-phase commit-reveal process, comprising:

broadcasting a blinded commitment hash comprising the extracted private key, a submitter wallet address, and a salt; and

subsequently revealing the extracted private key and salt, wherein a distributed ledger smart contract disburses a discovery bounty exclusively to the submitter wallet address.

5. The method of claim 1, wherein the client computing process increments the monotonic state variable h using an affine non-copyable linear type enforced by a compiler, preventing duplicate invocation of the signature routine at compile time.

6. The method of claim 1, wherein the distributed ledger smart contract enforces a graduated liquidated damages penalty, calculating the foreclosed collateral as a function of the actual un-settled computational resources rather than forfeiting the entirety of the collateral parameter B .

7. The method of claim 1, wherein intermediate bilateral debts between a plurality of client computing processes are periodically cancelled off-chain via directed cycle cancellation on a peer-to-peer debt graph prior to distributed ledger finalization.

8. A distributed computing system for high-frequency peer-to-peer machine-to-machine streaming settlements, comprising:

a service provider hardware processor;

a local memory coupled to the processor storing instructions that, when executed, configure the processor to:

maintain a localized un-settled exposure threshold δ_v ;

receive streaming cryptographic micro-cheques bound to pre-committed nonces over a transport layer without querying a distributed ledger;

halt service delivery upon reaching the threshold δ_v when the global on-chain collateral parameter B satisfies $B > \sum \delta_v$; and

extract an equivocation private key using modular arithmetic subtraction and inversion operations

to generate a collateral foreclosure transaction.

9. A non-transitory computer-readable medium storing instructions for executing the method of claim 1.
10. The non-transitory computer-readable medium of claim 9, wherein the instructions further configure a client processor to derive deterministic Schnorr nonces via a keyed pseudorandom function $k_h = \text{HMAC}(sk, h) \bmod q$.

ABSTRACT OF THE DISCLOSURE

A system, method, and computer-readable medium are disclosed for drastically reducing network latency and memory bottlenecks in asynchronous machine-to-machine streaming computational micro-transactions. A client computing process initializes a performance collateral parameter B on a base distributed ledger and anchors a Merkle root of deterministic Schnorr nonces indexed by a monotonic state variable h . Micro-payments are streamed directly peer-to-peer with zero network roundtrip latency. Service providers enforce a localized volatile memory buffer cap $\Delta_v \leq \$1.00$, wherein global solvency requires $B > \sum \Delta_v$. If a client attempts equivocation by issuing conflicting cheques on the same height h , the observing provider extracts the client's private key via localized modular arithmetic operations in under 25 microseconds and broadcasts a fraud-proof transaction triggering automated collateral foreclosure for liquidated damages.